Birzeit University - Faculty of Engineering and Technology - Department of Computer Science
Artificial Intelligence (COMP338) - Course Project, Summer Semester 2025/2026

AI-Based University Course Scheduling Using a Genetic Algorithm

Student: Mohammad Humaidan (1231747) **Instructor:** Abdelrahman Hamarsha **Section:** 1

1. Introduction

Every semester a university has to give each course a day, a time and a room. The timetable is only usable if no instructor, room or student group is booked twice at the same time, every room is big enough and of the right type, and no instructor teaches when unavailable. A good timetable also avoids very early and very late classes and keeps students' days compact.

Even a small problem is huge: every course has 120 possible positions, so 30 courses can be arranged in $120^{30} \approx 10^{62}$ ways. Trying them all is impossible and doing it by hand is slow and error-prone, so an AI search method is used. A **Genetic Algorithm (GA)** [1, 2] keeps a population of complete timetables and improves it over many generations with selection, crossover and mutation.

Objectives: model the problem with hard and soft constraints; implement every GA component from scratch in Python (no GA library); produce and display a timetable without conflicts; and measure how population size, mutation rate, crossover rate and problem size affect the result, comparing the GA with hill climbing.

2. Problem Formulation

Time slots and rooms. Five days (Sunday-Thursday) with four one-hour periods (08:00-12:00) give 20 time slots. The six rooms of the brief - R101 (50 seats), R102 (40), R103 (35), R104 (25), LAB1 (30) and LAB2 (25) - give 20 × 6 = 120 positions.

Courses, instructors and student groups. Each course has a number of students, a fixed instructor, a type (lecture or lab) and the student groups that attend it, and meets once a week. The first 10 courses are exactly the dataset of the brief. Larger datasets (20, 30, 40 and 50 courses) add real computer-science courses, up to 20 instructors with unavailable times and preferred periods, and up to 14 student groups such as CS-Y3; some courses are shared by two groups. The main dataset has 30 courses, 15 instructors and 11 groups.

Table 1: Hard constraints (H, must always hold) and soft constraints (S, preferences)

ID	Constraint	Violated when
H1	Instructor conflict	an instructor has two courses at the same time
H2	Room conflict	a room has two courses at the same time
H3	Student-group conflict	a student group has two courses at the same time
H4	Room capacity	the course has more students than the room has seats
H5	Room type	a lab course is not in a lab, or a lecture is in a lab
H6	Instructor availability	the instructor is unavailable at that time (e.g. Dr. Ahmad, Monday 10:00)
S1	Early classes	a course is at 08:00
S2	Late classes	a course is at 11:00 (the last period)
S3	Student gaps	a group has an empty period between two of its classes on the same day
S4	Instructor preferences	a course is outside its instructor's preferred periods
S5	Working days	a group has classes on one more day (fewer days is better)

Fitness function. As in the brief, the penalty adds up the hard (H) and soft (S) violations, and the fitness turns it into a score where higher is better:

$$\text{Penalty} = \text{WH} \times \text{H} + 10 \times \text{S} \qquad \text{Fitness} = 1 \,/\, (1 + \text{Penalty})$$

WH is set higher than the largest possible soft penalty (WH = 2450 for 30 courses), so a timetable without hard violations always beats one with a clash. With H = 0 the examples of the brief are reproduced exactly: S = 10 gives 1/101 = 0.0099 and S = 3 gives 1/31 = 0.0323.

3. Genetic Algorithm

Table 2: The parts of the genetic algorithm

Part	What the program does	Why
Chromosome	one complete timetable: one gene per course, always in the same course order	every course is scheduled exactly once
Gene	(day, period, room), e.g. (1, 2, 1) = Monday 10:00 in R102	the brief's Gene = (Day, Time, Room)
Initial population	100 random timetables; a course only gets positions that fit its size, room type and instructor	no time is wasted on impossible positions
Fitness	the penalty formula above, counting all 11 constraints	lower penalty = better timetable
Selection	tournament: pick 4 random timetables, the best one becomes a parent (roulette wheel is also available)	simple, and not affected by the size of the fitness values
Crossover	two-point: swap the middle block of courses between two parents, probability 0.8 (one-point is also available)	combines good parts of two timetables; children are always valid
Mutation	each course moves to a new random position with probability 0.05 (swap, change time and change room are also available)	keeps exploring new timetables
Elitism	the best 5 timetables are copied unchanged into the next generation	the best timetable is never lost
Termination	stop after 500 generations, after 50 generations without improvement, or at a target soft score	stop when the search stops improving

Each generation evaluates every timetable, remembers the best one found so far, and builds the next population from the elites plus mutated children of selected parents, until a termination condition is met. For comparison, **hill climbing** starts from one random timetable, moves one random course at a time and keeps the change if the timetable does not get worse.

4. Code Explanation

The program is written in Python. `main.py` contains the whole program, `experiments.py` runs all experiments and draws the graphs, and `test_project.py` contains 37 unit tests that check every constraint and operator. The most important parts of `main.py` are:

Table 3: Main classes and functions of main.py

Part	Purpose
Course, Instructor, Room, Problem	hold the input data (built-in datasets or a JSON file)
Evaluator	the fitness function: counts every hard and soft violation and returns the penalty and fitness
initialize_population	creates the random starting timetables
tournament_selection, roulette_wheel_selection	choose the parents
two_point_crossover, one_point_crossover	create two children from two parents
random_mutation, swap_mutation, change_time_mutation, change_room_mutation	make small random changes
Config, genetic_algorithm	the GA settings and the main GA loop
hill_climbing	the comparison algorithm

Part	Purpose
SchedulerApp	the desktop window that shows the timetable, the fitness graph and the constraint counts

Running `main.py` opens the desktop application (Figure 1): choose a dataset and the GA settings, click *Generate timetable*, and the program shows the timetable, a weekly grid, the fitness graph and the constraint counts. `python main.py --cli` prints the same result in the console.

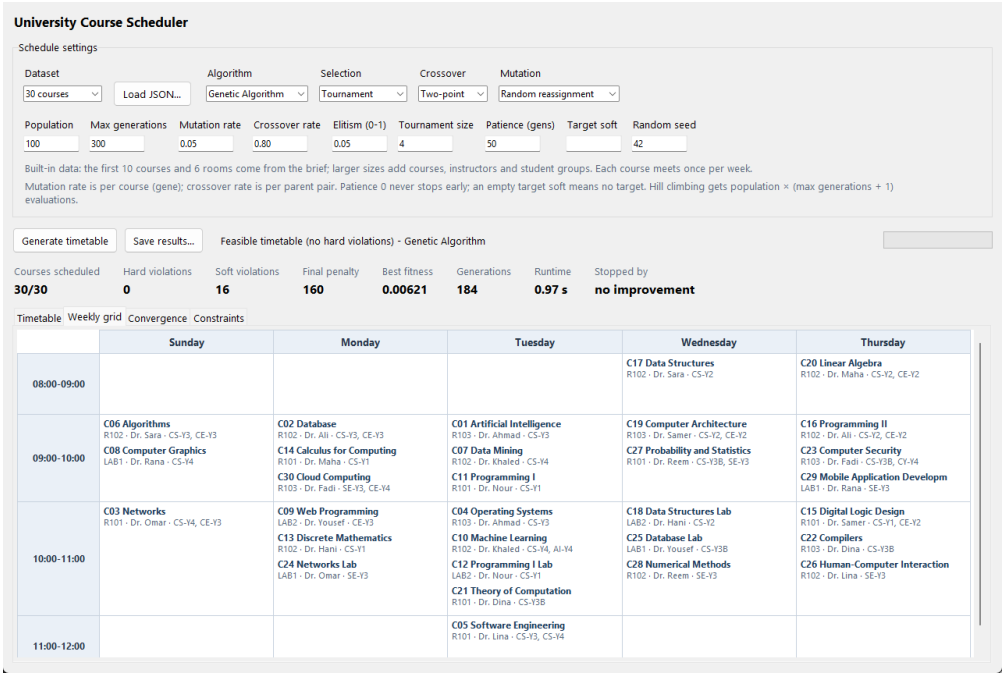


Figure 1: The desktop application showing the final 30-course timetable

5. Experimental Work

All experiments use the 30-course dataset and change one setting at a time from the default (population 100, mutation 5%, crossover 80%, 500 generations). Every setting was run with 10 different random seeds and the results were averaged. All 210 GA runs found a timetable with **zero hard violations**, so the table compares the remaining soft violations (lower is better).

Table 4: Experimental results, mean of 10 runs per setting

Experiment	Settings compared	Soft violations (mean)	CPU time (s)
1. Population size	20 / 50 / 100 / 200	17.6 / 17.2 / 17.0 / 16.4	1.2 / 2.6 / 5.4 / 11.7
2. Mutation rate	1% / 5% / 10% / 20%	17.8 / 17.0 / 18.0 / 26.4	5.3 / 5.4 / 5.6 / 5.7
3. Crossover rate	60% / 80% / 90%	16.8 / 17.0 / 16.2	5.4 / 5.4 / 5.1
4. Dataset size	10 / 20 / 30 / 40 / 50 courses	5.1 / 11.1 / 17.0 / 23.9 / 33.8	2.3 / 4.4 / 5.4 / 7.4 / 8.3
A. Selection	tournament / roulette wheel	17.0 / 17.3	5.4 / 6.0
B. Crossover method	one-point / two-point	16.9 / 17.0	5.7 / 5.4
C. Mutation method	random / change time / swap / change room	17.0 / 17.4 / 18.4 / 40.5	5.4 / 5.2 / 6.0 / 5.2
D. GA vs hill climbing	GA / hill climbing (same number of evaluations)	17.0 / 14.5	5.4 / 2.2

What the results show:

- **Population size:** a bigger population tries more timetables per generation, so the result improved a little, but the time grew in proportion (almost ten times from 20 to 200). 100 is a good balance.

- **Mutation rate:** the most important setting. With 1% the timetables in the population quickly become almost identical, so the search gets stuck early (premature convergence). With 20% about 6 of 30 courses are moved in every child, which destroys good timetables and makes the search almost random (26.4 soft violations). 5% worked best (Figure 2).
- **Crossover rate:** 60%, 80% and 90% gave almost the same results.
- **Dataset size:** every size up to 50 courses was solved without hard violations. The time grows roughly linearly, but more courses must share the same 120 positions, so the soft violations grow.
- **Selection and crossover method (A, B):** tournament and roulette wheel, and one-point and two-point crossover, gave almost the same results.
- **Mutation method (C):** "change room" never moves a course to another time, and all soft constraints depend on the time, so it gets stuck. Random reassignment was the best.
- **GA vs hill climbing (D):** hill climbing was better and faster on this problem, because improving a timetable mostly needs small one-course changes, which is exactly what hill climbing does. The GA spends many evaluations on children that are worse than their parents, but it is more general and could be combined with hill climbing.

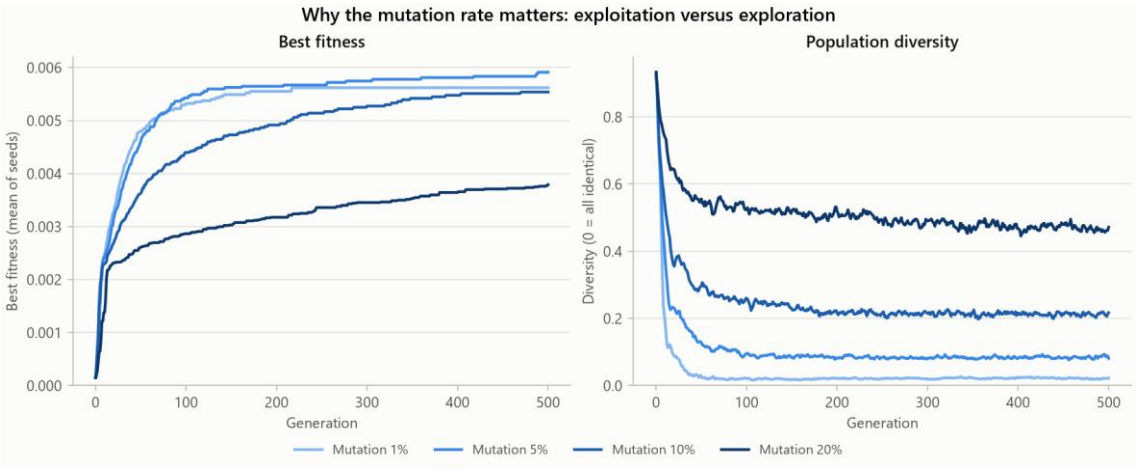


Figure 2: Mutation rate: best fitness (left) and how different the timetables in the population stay (right)

6. Results

Table 5: Results table in the format of the brief (mean of 10 runs, 30 courses)

Population	Mutation	Best fitness	Hard violations	Soft violations	CPU time
20	1%	0.00549	0	18.2	1.3 s
20	5%	0.00570	0	17.6	1.2 s
50	1%	0.00550	0	18.2	2.7 s
50	5%	0.00581	0	17.2	2.6 s
100	5%	0.00591	0	17.0	5.4 s
200	5%	0.00609	0	16.4	11.7 s

Larger populations give slightly better timetables but take more time, and with small populations 5% mutation is better than 1% because the population needs mutation to stay diverse.

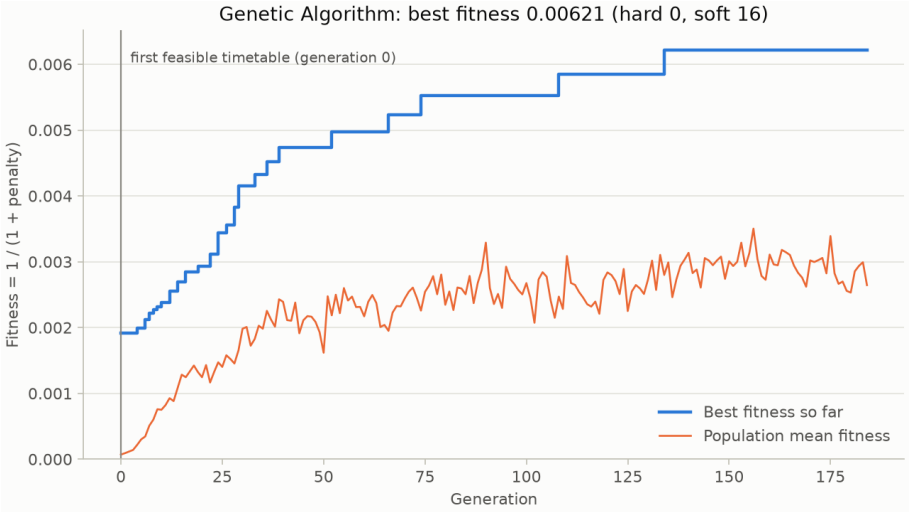


Figure 3: Generation vs best fitness for the run that produced the final timetable

Figure 3 shows that the GA really improves the solution: the best fitness never goes down (elitism) and rises in steps each time a better timetable is found. The search stopped at generation 184 after 50 generations without improvement.

Table 6: Final timetable summary (default settings, 30 courses)

Courses scheduled	Hard violations	Soft violations	Final penalty	Best fitness	Generations
30/30	0	16	160	0.00621	184

Table 7: Final weekly timetable, 30 courses (default configuration, seed 42)

Time	Sunday	Monday	Tuesday	Wednesday	Thursday
08:00-09:00				**C17 Data Structures** R102 · Dr. Sara	**C20 Linear Algebra** R102 · Dr. Maha
09:00-10:00	**C06 Algorithms** R102 · Dr. Sara **C08 Computer Graphics** LAB1 · Dr. Rana	**C02 Database** R102 · Dr. Ali **C14 Calculus for Computing** R101 · Dr. Maha **C30 Cloud Computing** R103 · Dr. Fadi	**C01 Artificial Intelligence** R103 · Dr. Ahmad **C07 Data Mining** R102 · Dr. Khaled **C11 Programming I** R101 · Dr. Nour	**C19 Computer Architecture** R103 · Dr. Samer **C27 Probability and Statistics** R101 · Dr. Reem	**C16 Programming II** R102 · Dr. Ali **C23 Computer Security** R103 · Dr. Fadi **C29 Mobile Application Development** LAB1 · Dr. Rana
10:00-11:00	**C03 Networks** R101 · Dr. Omar	**C09 Web Programming** LAB2 · Dr. Yousef **C13 Discrete Mathematics** R102 · Dr. Hani **C24 Networks Lab** LAB1 · Dr. Omar	**C04 Operating Systems** R103 · Dr. Ahmad **C10 Machine Learning** R102 · Dr. Khaled **C12 Programming I Lab** LAB2 · Dr. Nour **C21 Theory of Computation** R101 · Dr. Dina	**C18 Data Structures Lab** LAB2 · Dr. Hani **C25 Database Lab** LAB1 · Dr. Yousef **C28 Numerical Methods** R102 · Dr. Reem	**C15 Digital Logic Design** R101 · Dr. Samer **C22 Compilers** R103 · Dr. Dina **C26 Human-Computer Interaction** R102 · Dr. Lina
11:00-12:00			**C05 Software Engineering** R101 · Dr. Lina		

7. Conclusion

- A complete genetic algorithm was implemented from scratch, with 6 hard and 5 soft constraints, all the required operators, and a desktop and console display of the timetable.
- It always produced timetables without conflicts (210 of 210 runs). The final 30-course timetable schedules all 30 courses with 0 hard and 16 soft violations.
- The mutation rate matters most (5% was best). Larger populations help a little but cost time, and the crossover settings matter little.

- On this problem hill climbing was better and faster than the GA.
- Future improvements: combine the GA with hill climbing, change the mutation rate automatically, allow courses that meet more than once a week, and test with real university data.

References

[1] J. H. Holland, Adaptation in Natural and Artificial Systems. University of Michigan Press, 1975.

[2] D. E. Goldberg, Genetic Algorithms in Search, Optimization, and Machine Learning. Addison-Wesley, 1989.

[3] COMP338 Course Project brief, Birzeit University, Summer Semester 2025/2026.